Overview:

For lab three, I implemented a parser, which connects my scanner from lab one, to my AST printer from lab two.

Extra Credit Request:

For my language, I also implemented several features that go beyond a basic implementation. During this lab, I implemented advanced error diagnostics, which include the column number as well as the line. My design also detects the error of missing left operand in a binary expression in the primary function. I also made my parser support multiple single-line inputs, which differs from how the textbook integrated the parser, and how the instructions describe it. My method only handles this behaviour in execute() in lu.py, rather than the parser itself, so it will be removed in a later lab, thus it assumes each expression fits on one line. This addition took some extra work to ensure that the parser works as expected, and only works on one line as expected, but it also adds the benefit of larger test files, since I am able to add all of my test cases to one file. I did test the functionality both before and after this change, by calling the parser directly first, so I am confident that this change was a helpful intermediate step.

Syntactical Grammar Design:

My language currently uses the same syntactical grammar design as Lox, as defined below. I chose to retain this grammar because Lab 3 focuses on the translation from scanned tokens into an AST, so I have not yet made many choices that can stray from the original lox design.

expression → equality ;

equality → comparison (("!=" | "==") comparison)* ;

comparison → term ((">" | ">=" | "<" | "<=") term)* ;

term → factor (("-" | "+") factor)* ;

factor → unary (("/" | "*") unary)* ;

unary → ("!" | "-") unary

| primary ;

primary → NUMBER | STRING | "true" | "false" | "nil"

| "(" expression ")" ;

My language follows Lox's standard operator precedence, where grouping and literals are handled at the primary level, followed by unary operators (!, -), multiplication and division (*, /), addition and subtraction (+, -), comparison operators (<, <=, >, >=), and equality operators (==, !=). Operators with higher precedence are evaluated first when constructing the AST. Binary operators at the same precedence level are left-associative. Unary operators are

right-associative. Parentheses can override the default precedence rules by explicitly grouping expressions.

Setup Instructions & Design Explanation:

My test files are under test\lab3\ and can therefore be run in the following format from the project root:

python .\src\lu.py .\test\lab3\[test_file.lu]

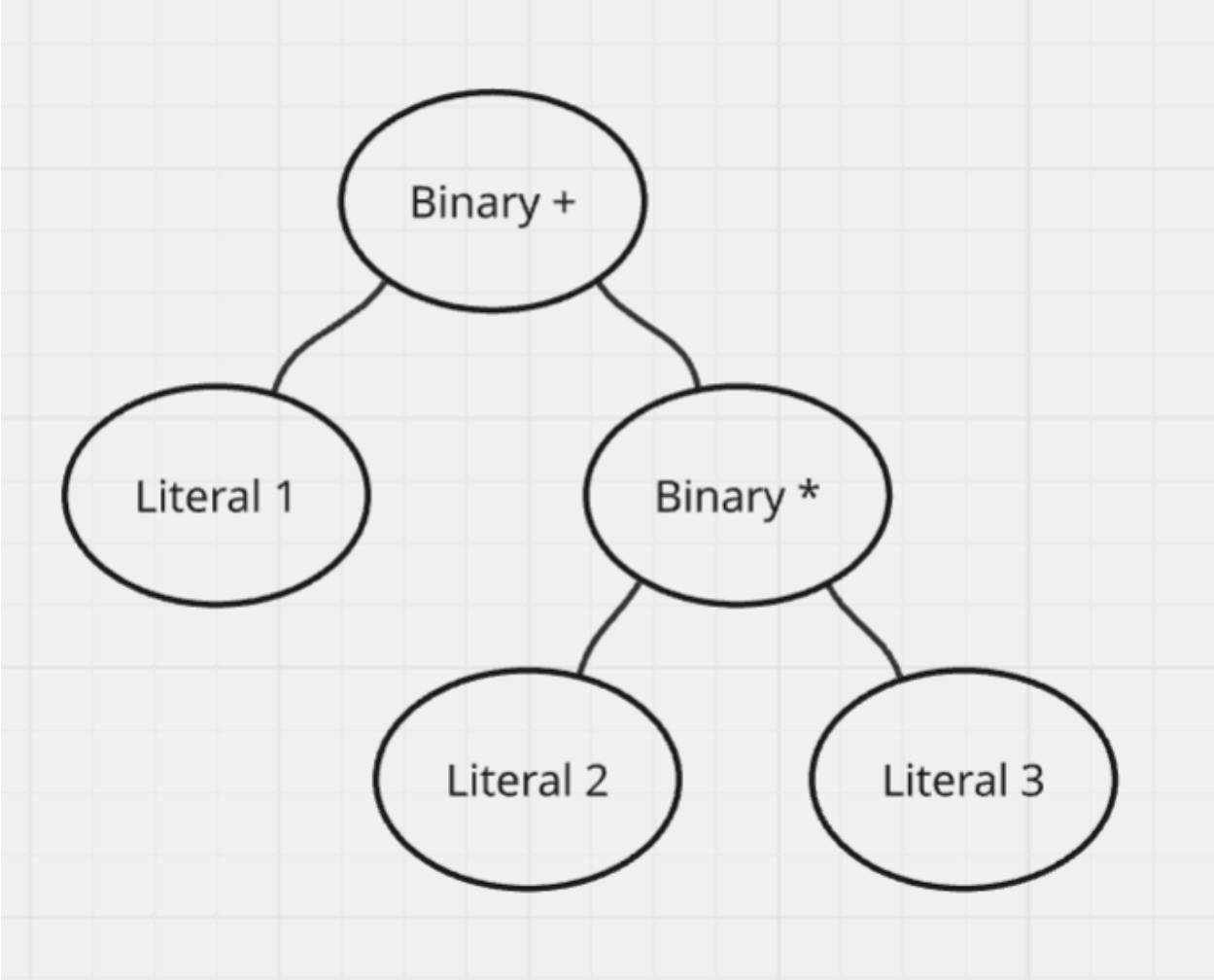
The repl mode still works similarly, by running:

python .\src\lu.py

Both modes accept valid lu-syntax code and print an AST as a result.

Syntax Tree:

The following tree represents the tree for 1 + 2 * 3. The program's printed AST is: (+ 1.0 (* 2.0 3.0)), which is the prefix notation of this tree.



Limitations:

All implemented test cases produced the expected output. The syntactical grammar is currently near-identical to Lox, since I heavily referenced its structure during my implementation. More nuances to syntax will likely emerge as I implement more features. Currently, running the files takes all input and returns either a printed AST or an error.

Also, the execute() function currently parses each line as a separate expression. This makes testing multiple expressions in one file easier, but means expressions that span multiple lines won't work properly. In the future, I could improve this by having the parser handle the entire input instead of splitting it by line.

Testing:

For all of the following test cases, input is provided as valid lu-syntax code and the outputs are printed ASTs.

Specific descriptions of the test cases are provided below, and all expected outputs match the actual outputs which are shown below. All test cases are defined in the files listed under .\test\lab3\, and their outputs are shown below. All expected outputs match the actual outputs.

Test Parser:

This round of test cases tests all valid output types, and ensures that the associativity and precedence is followed according to the syntactical rules defined previously. All test cases pass as expected and return the proper AST printed format.

[line 1] 42

42.0

[line 2] true

true

[line 3] false

false

[line 4] nil

nil

[line 6] -5

(- 5.0)

[line 7] !true

(! true)

[line 8] !!true

(! (! true))

[line 10] 1 + 2

(+ 1.0 2.0)

[line 11] 1 - 2

(- 1.0 2.0)

[line 12] 2 * 3

(* 2.0 3.0)

[line 13] 6 / 2

(/ 6.0 2.0)

[line 14] 1 == 2

(== 1.0 2.0)

[line 15] 1 != 2

(!= 1.0 2.0)

[line 16] 1 < 2

(< 1.0 2.0)

[line 17] 1 <= 2

(<= 1.0 2.0)

[line 18] 1 > 2

(> 1.0 2.0)

[line 19] 1 >= 2

(>= 1.0 2.0)

[line 21] 1 + 2 * 3

(+ 1.0 (* 2.0 3.0))

[line 22] 8 - 3 - 1

(- (- 8.0 3.0) 1.0)

[line 24] (1 + 2) * 3

(* (group (+ 1.0 2.0)) 3.0)

[line 25] (1 + (2 * 3))

(group (+ 1.0 (group (* 2.0 3.0))))

[line 27] 1 + 2

(+ 1.0 2.0)

[line 28] 3 * 4

(* 3.0 4.0)

Test Parser Errors:

The outputs for these test cases display the line number twice, because my temporary execute() sends each source line to the scanner separately, so the scanner sees each expression as a new one-line input. This is expected for my current testing approach, rather than a parser-specific bug. All outputs match the expected and are as follows.

[line 1] 1 +

[line 1, column 4] Error at end: Expect expression.

This expression is invalid because the parser expects the right operand.

[line 2] (1 + 2

[line 1, column 7] Error at end: Expect ')' after expression.

This expression is invalid because the parser expects a closing parenthesis.

[line 3] !

[line 1, column 2] Error at end: Expect expression.

This expression is invalid because the parser expects an expression to follow the bang.

[line 4] 1 2

[line 1, column 3] Error at '2': Unexpected token after expression.

This expression is invalid because the parser doesn’t expect to take in two literals without any operand.

[line 5] + 1

[line 1, column 1] Error at '+': Missing left operand for '+'.

This expression is invalid because the parser expects the left operand.